Linux Shell Scripting – Basic Interview & Revision Notes

---

1. What is Linux Shell?

- Linux Shell is an **interface** between:
  - User
  - Operating System (Kernel / Hardware)
- It performs the following functions:
  - Accepts commands from the user
  - Interprets the commands
  - Executes them by interacting with the kernel
- Shell acts as a **command interpreter**.
- Without a shell, users cannot directly interact with the kernel.

Flow:

- User
  - → Shell
    - → Kernel
      - → Hardware

---

2. Applications of Shell

Shell is mainly used for:

- File and directory manipulation (create, delete, modify files)
- Program execution
- User interaction (input/output)
- Task automation (backup, cleanup, monitoring)
- System administration tasks (user management, service control)

---

3. What is a Shell Script?

Akash Khandagale

- A shell script is:
  - A file containing a **set of Linux commands**
- Execution behavior:
  - Commands run **sequentially (line by line)**
- Purpose:
  - Automate repetitive tasks
  - Reduce manual work
  - Ensure consistency in operations
- Shell scripts save time and reduce human error.

---

4. Shell Script Format

```
#!/bin/bash
```

- `#!` → Shebang
- `/bin/bash` → Bash shell interpreter
- This line tells the system:
  - Which shell should execute the script
- Mandatory for executing scripts using `./script.sh`.

---

5. Types of Shells

Commonly used shells in Linux:

- **Bourne Shell (sh)** – Original Unix shell
- **Bourne Again Shell (bash)** – Default shell in most Linux systems
- **Korn Shell (ksh)** – Enhanced scripting features
- **C Shell (csh)** – C-like syntax
- **TC Shell (tcsh)** – Improved version of csh

---

6. Echo Command

- Purpose:
  - Display output on the terminal

Akash Khandagale

- Commonly used for:
  - Debugging scripts
  - Displaying messages to users

```
echo "Hello World"
```

---

7. Check Type of Shell

- Used to identify the current shell being used
- Helpful in troubleshooting shell-related issues

```
echo $SHELL
```

---

8. Variables in Shell Script

Key Points

- Variables are **case-sensitive**
- No space is allowed before or after `=`
- Variables store temporary data used by scripts

Syntax

```
variable_name=value
```

Example

```
host=$(hostname)
echo $host
```

---

9. Taking Input from User

- `read` command is used to take input from user at runtime
- Helps make scripts interactive

Akash Khandagale

Syntax

```
read variable_name
```

Example

```
echo "Enter your name:"
read name
echo "My name is $name"
```

---

10. If-Else Conditional Statement

Purpose

- Used for **decision making** in scripts
- Allows execution of commands based on conditions

Syntax

```
if [ condition ]
then
   commands
else
   commands
fi
```

Example: Voting Eligibility

```
echo "Enter your age:"
read age

if [ $age -ge 18 ]
then
  echo "You are eligible to vote"
else
  echo "Sorry, you are not eligible to vote"
fi
```

Akash Khandagale

11. Comparison Operators (Numeric)

Operator	Meaning
`-eq`	Equal to
`-ne`	Not equal to
`-gt`	Greater than
`-lt`	Less than
`-ge`	Greater than or equal to
`-le`	Less than or equal to

- These operators are mainly used inside `if` conditions.

Linux Shell Scripting – Execution, Loops, Functions & Special Variables

---

12. Executing a Shell Script

12.1 Steps to Execute

1. First check the **file permissions** of the script.
2. If execute permission is not present, assign it using `chmod`.
3. If execute permission is not available, the script can still be executed using `bash`.
4. Execute permission is required when running scripts using `./script.sh`.

12.2 Commands

```
chmod +x script.sh
./script.sh
```

12.3 Execute Without Permission

```
bash script.sh
```

- Useful when you do not have execute permission on the file.

Akash Khandagale

13. Switch Case Statement

13.1 Purpose

- Used when there are **multiple choices**.
- Cleaner and more readable than multiple `if-else` conditions.
- Commonly used in **menu-driven scripts**.

13.2 Syntax

```
case $choice in
  option1) commands ;;
  option2) commands ;;
  *) default command ;;
esac
```

13.3 Example

```
echo "Choose an option"
echo "a) Show current date"
echo "b) List files in current directory"
read choice

case $choice in
  a) date ;;
  b) ls ;;
  *) echo "Not a valid input" ;;
esac
```

Akash Khandagale

---

14. For Loop

14.1 Purpose

- Used to repeat a set of commands multiple times.
- Helps in automating repetitive tasks.

14.2 Numeric For Loop

```
for i in 1 2 3 4 5
do
  echo "Number is $i"
done
```

14.3 Loop Over Strings

```
for j in eat play sleep
do
  echo "$j"
done
```

14.4 Range-Based Loop

```
for i in {1..10}
do
  echo $i
done
```

---

15. Comments in Shell Script

15.1 Single-Line Comment

- Used to explain code logic

```
# This is a single-line comment
```

15.2 Multi-Line Comment

- Used to disable blocks of code or add documentation

```
: << 'COMMENT'
This is a
multi-line
comment
COMMENT
```

Akash Khandagale

16. While Loop

16.1 Purpose

- Executes commands **as long as the condition remains true**.
- Useful when the number of iterations is not fixed.

16.2 Example

```
count=0
num=10

while [ $count -le $num ]
do
  echo "Number is $count"
  let count++
done
```

---

17. Iterating Values from a File

17.1 Purpose

- Used to read values line by line from a file.
- Commonly used in log processing and configuration handling.

17.2 Example

```
hosts="/path/to/file"

for item in $(cat $hosts)
do
  echo $item
done
```

---

18. Arithmetic Expressions Using `let`

18.1 Purpose

- Used to perform arithmetic operations in shell scripting.

AkashKhandagale

- Mainly used with integer values.

18.2 Examples

```
let a=5*10
let a++
```

---

19. Alias

19.1 Purpose

- Alias is a **shortcut** for a long command.
- Improves productivity.

19.2 Syntax

```
alias short_name='command'
```

19.3 Examples

```
alias ll='ls -ltr'
alias pscheck='ps -ef | grep tomcat'
```

19.4 Display All Aliases

```
alias
```

---

20. Display Script Name

20.1 Using `$0`

- `$0` stores the name of the script being executed

```
echo "Script name is: $0"
```

20.2 Using `basename`

Akash Khandagale

- Removes directory path and shows only the script name

```
basename /home/user/scripts/myscript.sh
```

Output:

```
myscript.sh
```

---

21. Functions in Shell Script

21.1 Definition

- A function is a **block of reusable code**.
- Helps reduce code duplication.
- Improves script readability and maintainability.

21.2 Method 1

```
myfun() {
  echo "Hello from function"
}
```

21.3 Method 2

```
function myfun {
  echo "Hello"
}
```

21.4 Calling a Function

```
myfun
```

---

22. Function with Arguments

22.1 Example

Akash Khandagale

```
addition() {
  local num1=$1
  local num2=$2
  let sum=num1+num2
  echo "Sum of $num1 and $num2 is $sum"
}

addition 10 20
```

- `$1` , `$2` represent arguments passed to the function.

---

23. Special Shell Variables

Variable	Description
`$RANDOM`	Generates a random number (0–32767)
`$UID`	User ID of the logged-in user

---

24. Random Number Example (Dice Game)

```
#!/bin/bash
NUM=$((RANDOM % 6 + 1))
echo "Dice result: $NUM"
```

---

25. Check if Script is Executed by Root User

25.1 Concept

- Root user has **UID = 0**.
- Used to restrict script execution to administrators.

25.2 Example

```
if [ $UID -eq 0 ]
then
```

Akash Khandagale

```
  echo "You are root user"
else
  echo "You are not root user"
fi
```

Linux Shell Scripting – Redirection, Arguments & Utilities

---

26. Input / Output Redirection

26.1 Why Redirection is Used

Redirection is used in shell scripting when:

- You **don't want command output on the terminal**
- You want to **store command output in a file** for later use
- You want to **separate standard output and error output**
- Commonly used in **automation, logging, and cron jobs**

---

27. Types of Redirection

Symbol	Description
`>`	Redirect output (overwrite file)
`>>`	Redirect output (append to file)
`<`	Take input from file
`2>`	Redirect error output
`&>`	Redirect both output and error

---

28. Redirect Command Output to File

28.1 Overwrite Output

Akash Khandagale

- Existing file content will be replaced

```
pwd > out.txt
```

28.2 Append Output

- New output is added at the end of the file

```
pwd >> out.txt
```

---

29. Redirect Errors to File

29.1 Purpose

- Used to store error messages separately
- Helpful for troubleshooting

29.2 Example

```
cd /root 2> error.txt
```

---

30. Reading Input from File Inside Script

30.1 Purpose

- Used to read file content directly into a variable
- Helpful when processing configuration files

30.2 Example

```
#!/bin/bash
FILE="/home/paul/scripts/file.txt"
read TEXT < $FILE
echo $TEXT
```

Akash Khandagale

31. What is /dev/null?

31.1 Explanation

- `/dev/null` is a **special system file**
- Discards everything written to it
- Often called a **black hole** in Linux

31.2 Use Cases

- Suppress unwanted output
- Ignore errors in scripts

31.3 Example

```
cd /root &> /dev/null
```

---

32. Arguments in Shell Script

32.1 Purpose

- Arguments allow scripts to accept input at runtime
- Makes scripts dynamic and reusable

32.2 Running Script with Arguments

```
./myscript.sh arg1 arg2 arg3
```

---

33. Accessing Script Arguments

Variable	Meaning
`$#`	Number of arguments
`$@`	All arguments
`$1, $2 ...`	Individual arguments

- Arguments are passed in order from left to right

Akash Khandagale

34. Example Script (Arguments)

34.1 Script

```
#!/bin/bash

echo "Number of arguments: $#"
echo "All arguments: $@"
echo "First argument: $1"
echo "Second argument: $2"
```

34.2 Run

```
./args-demo.sh Raju Sham
```

---

35. Shifting Arguments using shift

35.1 Concept

- `shift` removes the **first argument**
- Remaining arguments move one position to the left
- Useful when processing arguments sequentially

35.2 Example

```
#!/bin/bash

echo "Before shifting"
echo $@

shift

echo "After shifting"
echo $@
```

---

36. Practical Example – User Creation Script

Akash Khandagale

36.1 Script

```
#!/bin/bash

echo "User creation in progress"
echo "Name of the user is: $1"

shift

echo "User details: $@"
```

36.2 Run

```
./create-userdemo.sh Nick Nick Jones nick.jones@gmail.com
```

---

37. Loop Through All Arguments

37.1 Using For Loop

- Used to process each argument individually

```
for arg in $@
do
  echo "Argument: $arg"
done
```

---

38. Example – Print Names from Arguments

38.1 Script

```
#!/bin/bash

for name in $@
do
  echo "My name is $name"
done
```

38.2 Run

Akash Khandagale

```
./allargs-demo.sh Ram Sham Raju Baburao
```

---

39. sleep Command

39.1 Purpose

- Used to **pause execution** of a script
- Often used between two executions

39.2 Example

```
sleep 5
```

---

40. Stop Script Execution

40.1 exit Command

- Immediately terminates script execution

```
exit
```

---

41. Readonly Variable (Constant)

41.1 Explanation

- Once declared, the variable value **cannot be changed**
- Helps protect important variables

41.2 Example

```
readonly NAME="Akash"
```

---

42. Logging using logger (Syslog)

Akash Khandagale

42.1 Purpose

- Used to log messages from shell scripts
- Helpful for debugging and monitoring

42.2 Log Location

- Logs are stored in:
  - `/var/log/messages`

42.3 Example

```
logger "Hey Buddy"
```

42.4 View Logs

```
tail -f /var/log/messages
```

Akash Khandagale